

Carlos Eduardo Diniz de Almeida - RA 10444407
Tarik Nasser Ferreira Kanbour - RA 10435895

Multiplicação de Matrizes e Método de Monte Carlo - via Windows

	Tempo Sequencial (ms)	Tempo Multithread (ms)	Tempo distribuído MPI (ms)
N = 300	47758.12	40845.51	1845.01
N = 600	122638.24	72094	10843.63
N = 1000	529394.62	355323.45	37509.78

TEMPO N = 300

```
Successfully copied 4.1kB to master:/home/mpiuser/
Successfully copied 2.05kB to master:/home/mpiuser/
Successfully copied 4.1kB to worker1:/home/mpiuser/
Successfully copied 2.05kB to worker1:/home/mpiuser/
Successfully copied 4.1kB to worker2:/home/mpiuser/
Successfully copied 2.05kB to worker2:/home/mpiuser/
Successfully copied 4.1kB to worker3:/home/mpiuser/
Successfully copied 2.05kB to worker3:/home/mpiuser/
```

```
Warning: Permanently added 'worker3' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker1' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker2' (ED25519) to the list of known hosts.
Tempo sequencial: 47758.12 ms
Tempo sequencial: 48720.80 ms
Tempo sequencial: 48773.26 ms
Tempo sequencial: 48782.85 ms
Tempo com threads: 40845.51 ms
Tempo com threads: 41619.79 ms
Tempo com threads: 43522.36 ms
Tempo com threads: 42816.95 ms
Tempo distribuído: 1845.01
```

tempo = 600

```
kadue@LAPTOP-C9FLN2GP MINGW64 ~/OneDrive/Área de Trabalho/Mackenzie/Etapa 6/computacao-distribuida/computacao-distribuida/lab-3 (main)
$ python ex1.py
Tempo sequencial: 122638.24 ms
Tempo com threads: 72094.29 ms
```

```
Successfully copied 3.07kB to master:/home/mpiuser/
Successfully copied 2.05kB to master:/home/mpiuser/
Successfully copied 3.07kB to worker1:/home/mpiuser/
Successfully copied 2.05kB to worker1:/home/mpiuser/
Successfully copied 3.07kB to worker2:/home/mpiuser/
Successfully copied 2.05kB to worker2:/home/mpiuser/
Successfully copied 3.07kB to worker3:/home/mpiuser/
Successfully copied 2.05kB to worker3:/home/mpiuser/
```

```
Warning: Permanently added 'worker2' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker3' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker1' (ED25519) to the list of known hosts.
Tempo distribuído: 10843.63
```

tempo = 1000

```

Successfully copied 3.07kB to master:/home/mpiuser/
Successfully copied 2.05kB to master:/home/mpiuser/
Successfully copied 3.07kB to worker1:/home/mpiuser/
Successfully copied 2.05kB to worker1:/home/mpiuser/
Successfully copied 3.07kB to worker2:/home/mpiuser/
Successfully copied 2.05kB to worker2:/home/mpiuser/
Successfully copied 3.07kB to worker3:/home/mpiuser/
Successfully copied 2.05kB to worker3:/home/mpiuser/

Warning: Permanently added 'worker3' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker1' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker2' (ED25519) to the list of known hosts.
Tempo distribuído: 37509.78

```

```

$ python ex1.py
Tempo sequencial: 529394.62 ms
Tempo com threads: 355323.45 ms

```

Exercício 2 – Estimativa do Valor de π utilizando o Método de Monte Carlo - via Windows

	Tempo Sequencial (ms)	Tempo distribuído MPI (ms)
N = 10.000.000	1530.85	489.10

Tempo N = 10.000.000 sequencial:

```

tarik@tarik:/mnt/d/OneDrive/Tarik/Faculdade/6º Semestre/COMPUTAÇÃO DISTRIBUÍDA/Lab 3/
computacao-distribuida/lab-3$ python3 ex2_seq.py
Pi aproximado: 3.140981
Tempo experimental: 1530.85 ms

```

Tempo N = 10.000.000 distribuído:

```

IBUÍDA/Lab 3/computacao-distribuida/lab-3$ ./executar_python.sh
ex1.py ex2.py ex2_seq.py executar_python.sh programa.py setup
Successfully copied 930B (transferred 2.56kB) to master:/home/mpiuser/
Successfully copied 63B (transferred 2.05kB) to master:/home/mpiuser/
Successfully copied 930B (transferred 2.56kB) to worker1:/home/mpiuser/
Successfully copied 63B (transferred 2.05kB) to worker1:/home/mpiuser/
Successfully copied 930B (transferred 2.56kB) to worker2:/home/mpiuser/
Successfully copied 63B (transferred 2.05kB) to worker2:/home/mpiuser/
Successfully copied 930B (transferred 2.56kB) to worker3:/home/mpiuser/
Successfully copied 63B (transferred 2.05kB) to worker3:/home/mpiuser/

Warning: Permanently added 'worker1' (ED25519) to the list of known host
s.
Warning: Permanently added 'worker2' (ED25519) to the list of known host
s.
Warning: Permanently added 'worker3' (ED25519) to the list of known host
s.
Processos utilizados      : 4
Total de pontos (N)      : 10000000
Pontos dentro do circulo: 7855404
PI aproximado            : 3.142162
Tempo distribuido MPI    : 489.10 ms

```

Análise de Desempenho e Discussão de Resultados

Versão Multithread vs. Sequencial:

A versão multithreaded apresentou ganhos modestos (1.17x a 1.70x), especialmente para matrizes menores. Isso ocorre porque:

Overhead de sincronização: Threads compartilham o mesmo espaço de memória, mas a contenção por locks (mutex) em estruturas compartilhadas reduz o paralelismo efetivo.

GIL (Global Interpreter Lock): Em Python, o GIL limita a verdadeira paralelização em múltiplos núcleos para operações computacionais puras.

Overhead de criação de threads: Para matrizes pequenas (N=300), o custo de criar e sincronizar threads é comparável ao ganho obtido.

Versão MPI vs. Sequencial:

Os ganhos foram significativamente maiores (11.31x a 25.88x para matrizes), demonstrando que:

Paralelismo Real: MPI permite verdadeira execução paralela em múltiplos processos isolados (4 workers), contornando limitações como o GIL.

Escalabilidade: O speedup aumenta em problemas maiores (N=1000: 14.11x), indicando melhor distribuição de carga.

Eficiência: Para Monte Carlo com 10M pontos, o speedup de 3.13x é inferior ao esperado (ideal seria 4x com 4 processos), sugerindo overhead de comunicação.

Gargalos de Comunicação e Overhead - Overhead de Rede (MPI)

Serialização de Objetos: mpi4py serializa dados Python antes de transmiti-los entre processos. Para dados grandes, isso aumenta latência.

Comunicação Inter-container: A transmissão via rede entre containers (master, worker1, worker2, worker3) introduz latência significativa comparada a threads no mesmo processo.

Sincronização MPI: Operações de Allreduce() e Gather() exigem que todos os processos esperem pelo mais lento, criando gargalos em caso de carga desbalanceada.

Monte Carlo (N=10M): Speedup de 3.13x em 4 processos indica ~21% de perda de eficiência devido a overhead.

Matrizes (N=300): Speedup de 25.88x em 4 processos = eficiência de 97%, porém com matrizes pequenas o overhead absoluto é menor relativamente.

O maior gargalo ocorre em N=600 (matrizes) onde o speedup cai para 11.31x. Isso sugere um "sweet spot" onde dados suficientemente grandes para justificar overhead MPI, mas não tão grandes que a comunicação compense totalmente o trabalho computacional.